

Java Arrays: 1-D Array, 2-D Array and Jagged Array

A Conceptual and Practical Teaching Guide with Examples, Programs and Properly Commented Code

Learning Objectives

- Understand why arrays are required and how Java stores array elements.
- Declare, create, initialize, access, update and traverse one-dimensional arrays.
- Use arrays with Scanner, methods, searching, sorting and common calculations.
- Understand two-dimensional arrays as arrays of arrays and work with rows and columns.
- Understand jagged arrays and create rows having different lengths.
- Differentiate 1-D, 2-D and jagged arrays and select the appropriate structure for a problem.

1. Introduction to Arrays

An array is a fixed-size collection of elements of the same data type. The elements are stored and accessed using an index. Java arrays are objects, and their size is fixed after creation.

Why do we need arrays?

- They allow multiple values to be stored under one variable name.
- They make repetitive processing easier using loops.
- They are useful for searching, sorting, aggregation and data processing.
- They provide direct access to an element using its index.

Important characteristics

- Indexing starts from 0.
- For an array of length n, valid indexes are 0 to n-1.
- The length is available through the **length** property, not a method.
- Accessing an invalid index causes `ArrayIndexOutOfBoundsException`.
- The elements have default values when an array is created.

2. One-Dimensional (1-D) Array

A one-dimensional array represents a linear sequence of values. It is the most common array form and can be visualized as a row of elements.

Declaration, creation and initialization

```
// Declaration
int[] marks;

// Creation: creates an array that can store 5 integers
marks = new int[5];

// Declaration + creation
int[] scores = new int[5];

// Initialization using an array literal
int[] numbers = {10, 20, 30, 40, 50};
```

Accessing and updating elements

```
public class ArrayAccess {
    public static void main(String[] args) {
        int[] marks = {75, 82, 91, 68, 88};

        // Indexing starts at 0.
        System.out.println("First mark: " + marks[0]);
        System.out.println("Third mark: " + marks[2]);

        // Updating an element
        marks[3] = 72;

        System.out.println("Updated fourth mark: " + marks[3]);
        System.out.println("Array length: " + marks.length);
    }
}
```

3. Traversing a 1-D Array

Traditional for loop

```
public class TraverseArray {
```

```

    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Use index when we need the position of an element.
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Index " + i + " = " + numbers[i]);
        }
    }
}

```

Enhanced for loop

```

public class EnhancedFor {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // The loop directly gives each element.
        for (int value : numbers) {
            System.out.println(value);
        }
    }
}

```

Teaching point: Use the traditional loop when the index is required; use the enhanced for loop when only the values are needed.

4. Taking 1-D Array Input

```

import java.util.Scanner;

public class ArrayInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] numbers = new int[n];

        // Read each element from the user.
        for (int i = 0; i < numbers.length; i++) {
            System.out.print("Enter element " + i + ": ");
            numbers[i] = sc.nextInt();
        }

        // Display the entered array.
        System.out.println("Array elements:");
        for (int value : numbers) {
            System.out.print(value + " ");
        }

        sc.close();
    }
}

```

5. Common Operations on a 1-D Array

```

public class ArrayOperations {
    public static void main(String[] args) {
        int[] numbers = {12, 7, 25, 4, 18};

        int sum = 0;
        int min = numbers[0];
        int max = numbers[0];

        for (int value : numbers) {
            sum += value;

            if (value < min) {
                min = value;
            }

            if (value > max) {
                max = value;
            }
        }

        double average = (double) sum / numbers.length;
    }
}

```

```

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
        System.out.println("Minimum = " + min);
        System.out.println("Maximum = " + max);
    }
}

```

6. Searching in a 1-D Array

```

public class LinearSearch {
    public static void main(String[] args) {
        int[] numbers = {15, 23, 8, 42, 19};
        int target = 42;
        int position = -1;

        // Check every element until the target is found.
        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                position = i;
                break;
            }
        }

        if (position != -1) {
            System.out.println("Found at index: " + position);
        } else {
            System.out.println("Element not found");
        }
    }
}

```

7. Passing a 1-D Array to a Method

An array can be passed as an argument to a method. The method receives a reference to the array object, so changes to its elements are visible to the caller.

```
public class ArrayMethodDemo {  
    // Receives an array and prints all its elements.  
    static void display(int[] values) {  
        for (int value : values) {  
            System.out.print(value + " ");  
        }  
        System.out.println();  
    }  
  
    // Calculates and returns the sum of the array.  
    static int calculateSum(int[] values) {  
        int sum = 0;  
  
        for (int value : values) {  
            sum += value;  
        }  
  
        return sum;  
    }  
  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40};  
  
        display(numbers);  
  
        int result = calculateSum(numbers);  
        System.out.println("Sum = " + result);  
    }  
}
```

8. Returning a 1-D Array from a Method

```
public class ReturnArray {  
    // Creates and returns a new integer array.  
    static int[] createNumbers() {  
        int[] values = {10, 20, 30, 40, 50};  
        return values;  
    }  
  
    public static void main(String[] args) {  
        int[] result = createNumbers();  
  
        for (int value : result) {  
            System.out.print(value + " ");  
        }  
    }  
}
```

9. Two-Dimensional (2-D) Array

A two-dimensional array is commonly used to represent tabular data with rows and columns. In Java, a 2-D array is implemented as an array whose elements are themselves arrays.

```
// Declaration  
int[][] matrix;  
  
// Creation: 3 rows and 4 columns  
matrix = new int[3][4];  
  
// Declaration + initialization  
int[][] marks = {  
    {80, 75, 90},  
    {65, 88, 72},  
    {91, 84, 79}  
};
```

10. Accessing and Traversing a 2-D Array

```

public class TwoDArray {
    public static void main(String[] args) {
        int[][] matrix = {
            {10, 20, 30},
            {40, 50, 60},
            {70, 80, 90}
        };

        // matrix.length gives the number of rows.
        for (int i = 0; i < matrix.length; i++) {

            // matrix[i].length gives the number of columns in row i.
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }

            System.out.println();
        }

        // Access a particular element: row 1, column 2.
        System.out.println("Element = " + matrix[1][2]);
    }
}

```

11. Taking 2-D Array Input

```

import java.util.Scanner;

public class TwoDInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter rows: ");
        int rows = sc.nextInt();

        System.out.print("Enter columns: ");
        int columns = sc.nextInt();

        int[][] matrix = new int[rows][columns];

        // Outer loop controls rows.
        for (int i = 0; i < rows; i++) {

            // Inner loop controls columns.
            for (int j = 0; j < columns; j++) {
                System.out.print("Enter [" + i + "][" + j + "]: ");
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println("Matrix:");
        for (int[] row : matrix) {
            for (int value : row) {
                System.out.print(value + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

12. Row and Column Operations

```

public class MatrixOperations {
    public static void main(String[] args) {
        int[][] matrix = {
            {10, 20, 30},
            {40, 50, 60},
            {70, 80, 90}
        };

        // Calculate the sum of each row.
        for (int i = 0; i < matrix.length; i++) {
            int rowSum = 0;

            for (int j = 0; j < matrix[i].length; j++) {
                rowSum += matrix[i][j];
            }
            System.out.println("Row " + i + " sum = " + rowSum);
        }
    }
}

```

```
    }  
    // Calculate the sum of each column.  
    for (int j = 0; j < matrix[0].length; j++) {  
        int columnSum = 0;  
  
        for (int i = 0; i < matrix.length; i++) {  
            columnSum += matrix[i][j];  
        }  
  
        System.out.println("Column " + j + " sum = " + columnSum);  
    }  
}
```

13. Jagged Array

A jagged array is a two-dimensional array in which different rows can have different numbers of elements. Because Java represents a 2-D array as an array of row arrays, each row can be created with a different length.

```
public class JaggedArrayDemo {
    public static void main(String[] args) {

        // Create an array with 3 rows.
        int[][] data = new int[3][];

        // Each row has a different size.
        data[0] = new int[2];
        data[1] = new int[4];
        data[2] = new int[3];

        // Assign values.
        data[0][0] = 10;
        data[0][1] = 20;

        data[1][0] = 30;
        data[1][1] = 40;
        data[1][2] = 50;
        data[1][3] = 60;

        data[2][0] = 70;
        data[2][1] = 80;
        data[2][2] = 90;

        // Traverse the jagged array.
        for (int i = 0; i < data.length; i++) {
            for (int j = 0; j < data[i].length; j++) {
                System.out.print(data[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

14. Jagged Array Using Array Literals

```
public class JaggedLiteral {
    public static void main(String[] args) {

        // Every row can contain a different number of elements.
        int[][] marks = {
            {80, 75},
            {90, 85, 88},
            {70},
            {95, 91, 89, 93}
        };

        for (int i = 0; i < marks.length; i++) {
            System.out.print("Row " + i + ": ");

            for (int value : marks[i]) {
                System.out.print(value + " ");
            }

            System.out.println();
        }
    }
}
```

15. Practical Example: Student Marks

A jagged array is useful when different students have different numbers of subjects, courses, tests or activities.

```
public class StudentMarks {
    public static void main(String[] args) {

        // Student 1 has 3 subjects, student 2 has 4,
        // and student 3 has 2 subjects.
        int[][] marks = {
```



```

        {78, 85, 91},
        {66, 72, 80, 75},
        {90, 88}
    };

    for (int i = 0; i < marks.length; i++) {
        int sum = 0;

        for (int mark : marks[i]) {
            sum += mark;
        }

        double average = (double) sum / marks[i].length;

        System.out.println(
            "Student " + (i + 1) +
            " | Subjects = " + marks[i].length +
            " | Average = " + average
        );
    }
}

```

16. Difference Between 1-D, 2-D and Jagged Arrays

Feature	1-D Array	2-D Array	Jagged Array
Structure	Linear	Rows and columns	Rows with variable lengths
Example	int[] a	int[][] a	int[][] a
Access	a[i]	a[i][j]	a[i][j]
Typical use	List of values	Matrix/table	Unequal row data
Row lengths	Not applicable	Usually equal	Can differ

17. Important Array Concepts for Students

- **length:** For a 1-D array, array.length gives the number of elements. For a 2-D/jagged array, array.length gives the number of rows and array[i].length gives the size of a particular row.
- **Index:** Always starts from 0 and ends at length - 1 for each individual array.
- **Fixed size:** Once created, the size of a Java array cannot be changed. A new array is required for a different size.
- **Reference:** An array variable stores a reference to the array object.
- **Array of arrays:** A Java 2-D array is technically an array containing references to row arrays; this is what makes jagged arrays possible.
- **Null rows:** In a newly created jagged array such as new int[3][], the row references initially contain null until each row is created.
- **Bounds checking:** Accessing an invalid index results in ArrayIndexOutOfBoundsException.

18. Common Mistakes

- Using index <= array.length instead of index < array.length.
- Confusing array.length with a method call such as array.length().
- Using matrix.length for the number of columns in every row of a jagged array.
- Accessing a jagged row before initializing it.

- Assuming all rows of a Java 2-D array must have the same length.
- Forgetting that arrays are zero-indexed.

19. Suggested Lab Exercises

- Read n integers and print the largest and smallest values.
- Count even and odd numbers in a 1-D array.
- Search for an element and display all its positions.
- Reverse a 1-D array without using another array.
- Sort an array and compare manual sorting with `Arrays.sort()`.
- Add two matrices using 2-D arrays.
- Find the transpose of a matrix.
- Calculate row-wise and column-wise sums.
- Find the main diagonal sum of a square matrix.
- Create a jagged array for students having different numbers of subjects and calculate each student's average.
- Pass a 1-D or 2-D array to a method and return a computed result.
- Write a method that returns a new array containing only the even elements.

20. Quick Revision

1-D: one index $\rightarrow$ `array[i]`

2-D: two indexes $\rightarrow$ `array[i][j]`

Jagged: two indexes, but each row may have a different length $\rightarrow$ `array[i][j]`

Core idea: In Java, a 2-D array is an array of arrays. This concept is the foundation for understanding jagged arrays.